The tools and techniques of R programming
Lesson-5
The CASE structure in R

By
Prof.Dr. A.B.Chowdhury,HOD,CA

Techno India University, West Bengal,India
For any query,Email-Id:hod.ca@technoindiaeducation.com

August 28, 2023

Objective of this Lesson

This lesson deals with the **fourth fundamental logic structure used for structured programming which is known as CASE structure**. Through a considerable number of illustrative examples, an attempt has been made to make logic structure understandable to all keen readers.

Prerequisite: Knowledge of some procedural language programming will be useful.

The Case logic structure

A Case logic structure is the fourth acceptable kind of logic structure which is used in structured programming. Here, an expression or the value of an identifier is evaluated and then on the basis of its value, one of the mentioned values or value generated by one of the stated expressions will be returned.

This logic structure is used when one wishes to perform any one of a number of choices at a time. Each of the choices is called a case.

In some programming language, the word **'case'** is used explicitly, whereas in others, it remains implicit. For example, in C programming language, we can state any value numeric or non-numeric for each case by mentioning the keyword **'case'**; but in R, there is no such keyword and the numeric values from 1 to number of cases are to be used as subscripts of the identifier within square brackets where the returned value is stored. In case of non-numeric values, the mentioned values in the case structure are to be used. In R , the case structure is implemented through the **switch( ) function** of the system as shown below:

The syntax of the Case Structure in R

Identifier- to- hold- the- returned- value*assignment operator***switch (Variable/expression / value which is to be checked, [case-value-1=]value-1/expression-1,. . . . . .,[case-value-n=] value-n/ expression-n),**
The part within '[ ]' being optional.
Let us take a simple example to illustrate the syntax:

```
r=2
s=switch(r, 5,7,9)
```

Here, r[1]=5,r[2]=7 and r[3]=9.
The existing value of r is used as its subscript. So, s will receive:7; i.e., **print(s) will print 7**.
The preceding example shows the selection of the case value based on a Subscript like that in an array. Before studying further examples, let us first have a look at the pictorial form of the logic structure.
The flowchart structure of the logic is as shown below:

Pictorial structure of the logic

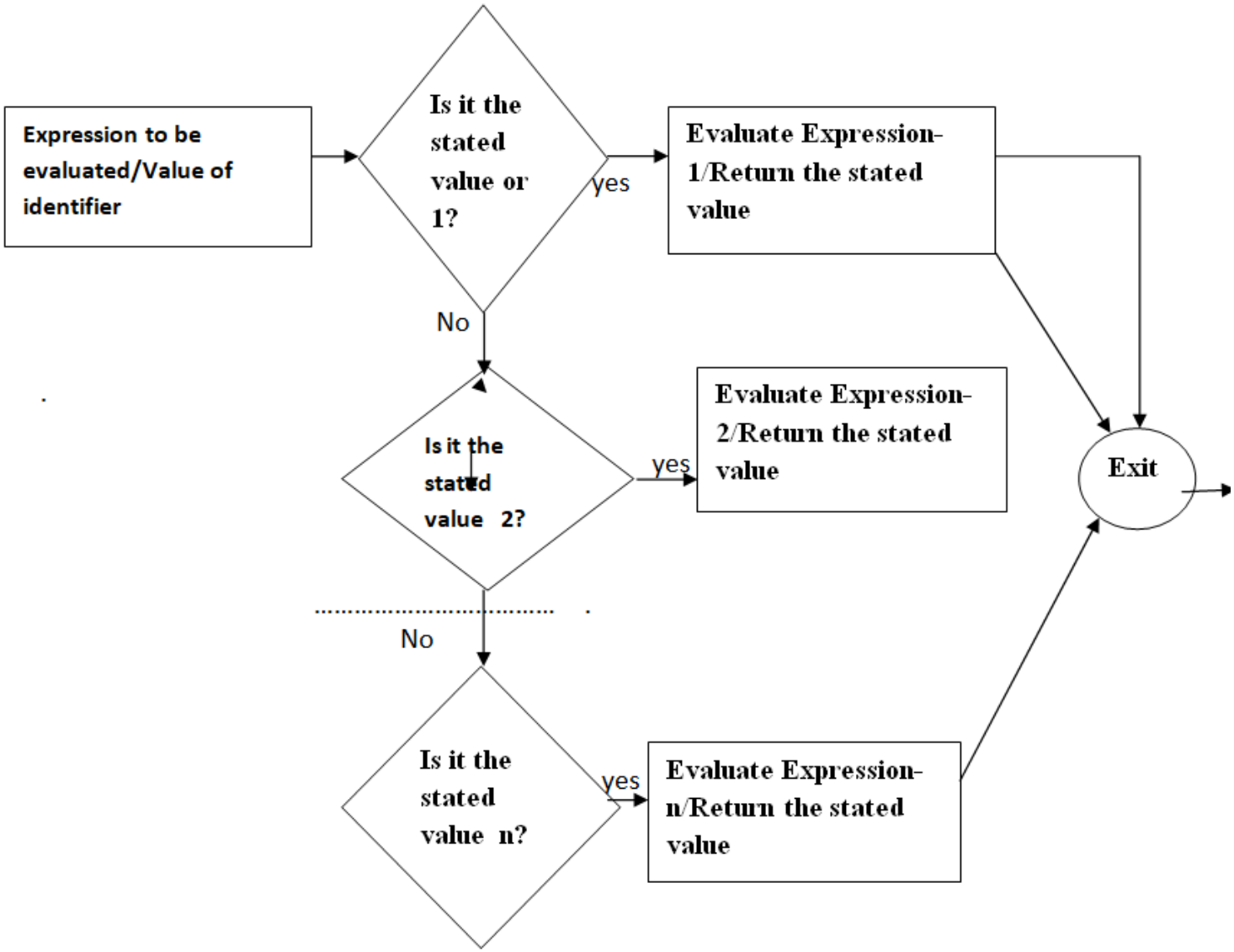

Figure: Pictorial presentation of 'Case' logic structure

A simple example of the R switch – case structure

Let us take another example of the R switch – case structure. We now take any arbitrary character value 5 in a variable as in the example shown below:

X="5"

Y=switch(X, "10"="Breakfast", "8"="Lunch", "5"="Dinner")

Now if we write: **print(Y)**, **the system will print "Dinner"**. The value of X can be varied depending on user's requirement. The return value will be based on matching value in this case. In place of values, we may write arithmetic or character expressions also.

Now, the following illustrative examples are meant for handling a variety of cases using the Case structure implementation in R in different generalized situations:

Illustrative example-5.1 using switch-Case logic

Example-5.1. Let us now develop a program that can be used as a desk calculator.
Task Analysis: We know that a desk calculator is used for basic arithmetic operations, such as, Addition (+), Subtraction (-), Multiplication (*) and real Division (/). We can state our choice either by specifying an operator or by mentioning an operation code,1, 2, etc. As any one of the operations is done at a time, the logic is ideally a case structure. The illustrative R script is shown below:

```
cat("Enter Any two real numbers")
n=scan(n=2);
cat("What would you like to Enter now? Type 1 for entering operator, 2 for entering operation code")
cat("Your Choice?")
v=scan(n=1)
switch(v,"Enter '+' to add,'-' to subtract,'*' for multiplication,'/' for real division,'%%'for modulo division,'%/%' for integer
division","1 for addition, 2 for subtraction,3 for multiplication,4 for real division")
ch=readline(prompt="Your option?")
y=switch(ch,
"+"=(n[1]+n[2]),
"-"=(n[1]-n[2]),
"*"=(n[1]*n[2]),
"/"=(n[1]/n[2]),
"1"=(n[1]+n[2]),
"2"=(n[1]-n[2]),"3"=(n[1]*n[2]),"4"=(n[1]/n[2]))

cat("The result is:",y)
```

Illustrative example-5.2 using switch-Case logic

Example-5.2.Develop an R script to print the number of days in each month.

Task Analysis: We know that the number of days in each month of a year is fixed except in a leap year when February takes 29 days.So, we remember the following verse:

Thirty days hath September,
April, June and November.
All the rest have thirty-one,
Excepting February alone,
And that has twenty-eight days clear
And twenty-nine in each leap year.

So, our R script can be as shown below. Once the R script starts execution, it will execute repeatedly until the condition of termination is satisfied.

```
# R Script to determine the number of days in a month of a year.
repeat {
cat("Enter a year of your choice to know the number of days in each its month")
cat("Enter 0 to terminate the script=>")
y=scan(n=1,quiet=TRUE)
if(y==0)
break
for(i in 1:12){
d=switch(i,31,ifelse((y%%4==0&&y%%100!=0||y%%400==0),29,28),31,30,31,30,31,31,30,31,30,31)
cat("Number of days in month number",i,"is:",d,")
}
}
```

Illustrative example-5.3 using switch-Case logic

Example-5.3. Let us develop an R script to show the name of the starting day of any given year.

Task Analysis: To determine the name of the starting day of any year, we need to know the day code of the starting month of the year i.e. January.The following formula is used for the purpose:

Day code=(year+$\lfloor \frac{year-1}{4} \rfloor$ - $\lfloor \frac{year-1}{100} \rfloor$**+**$\lfloor \frac{year-1}{400} \rfloor$**) mod 7**

Where $\lfloor x \rfloor$ implies the greatest integer contained in x.

Now, we evaluate the day code value as follows:

For day code=0, the day is **Sunday**;

For day code=1, the day is **Monday**;

For day code=2, the day is **Tuesday** and so on.

As for a given year, we shall have only one day code and hence only one name of a day, we may make use of the case structure logic. Again, the R case structure logic i.e. the **Switch() function** of the system cannot recognize the 0 value,hence,we increase the day code value by one forcibly and conclude accordingly. The following R script has been developed for the purpose:

R script for determining the name of the starting day of any given year

```
#Program to find out the starting day name of a given year
cat("Enter the year to check")
y=scan(n=1)
dc=(as.integer(y)+as.integer((y-1)/4)-as.integer((y-1)/100)+
as.integer((y-1)/400))%%7
dc=dc+1
day=switch(dc,"Sunday","Monday","Tuesday","Wednesday",
"Thursday","Friday","Saturday")
cat("The starting day of the year is:",day)
print("End of program....Thank you")
```

Illustrative example-5.4 using switch-Case logic

Problem 5.4. A Technical Institution assigns 7-digit student ID based on the basis of the following rules: First 3 digits identify the course of study of the student and the next 4 digits identify the student serial number. Considering the first 3 digits in the leading positions of the ID, the rule are: 100 implies 'BCA course', 101 implies 'Integrated MCA course', 200 implies 'M.Tech (CSE) course', 201 implies 'Integrated M.Tech (CSE) course', 301 implies 'M.Tech(EE) course', 302 implies 'Integrated M.Tech(EE) course'. Assuming that only these courses are conducted in the institution, an R script is required to be developed to determine the course of study on the basis of given student Id.

Task Analysis: Here, for a given student Id, we are required to separate first three digits from it to initiate the decision –making process. So, we shall use the **substring** function of R to do the task. The next stage of the logic will involve the 'case structure ' logic to draw the conclusion by selecting one of the many other alternatives. The syntax of the **substring()** function is: **substr(text, start, stop)**

Illustrative example-5.4 using switch-Case logic–CONTD.

where,

- **'text'** is a character string.
- **start** – If the characters of text were numbered, then the number of the first character to be returned (or overwritten).
- **stop** – The number of the last character to be returned (or overwritten).

An alternative syntax is:

substring(text, first, last = 1000000L)

Where,

- **'text'** is a character string.
- **first** – If the characters of 'text' were numbered, then the number of the first character to be returned (or overwritten).
- **last** – The number of the last character to be returned (or overwritten), which is to 1 million, by default.

In the solution,we may make use of **nchar(x)** to obtain the last character number in the **'text'**, where, **nchar()** returns the number of characters in

Illustrative example-5.4 using switch-Case logic–CONTD.

```
# String manipulation and application of switch()
s=readline(prompt="Enter the student ID to check")
scode=substr(s,1,3)
r=switch(scode,'100'='Student of BCA','101'='Student of Integrated MCA','200'='student of B.Tech(CSE)','201'= 'student of integrated M.Tech(CSE)','300'='Student of B.Tech(EE)','301'='Studentof integrated M.Tech(EE)','Invalid Code')
print( r )
```

Example 5. 5. It is required to develop an option-based system to calculate depreciation using any one of the following methods:

1. **Straight Line Method(SLM)**
2. **Declining Balance Method(DBM)**
3. **Sum of Years' Digits Method(SYDM)**

Illustrative example-5.5 using switch-Case logic

Task Analysis: Clearly, this is another problem to be solved using the 'case structure' logic. We know that, depreciation is the measurement in terms of money by which the cost of an article gets reduced owing to its use. The cost of the article is called its initial face value which gets reduced by the amount of depreciation at the end of each financial year.The useful period of the article in consideration is called its life. The number of financial years for which an article remains in use, is called period and the monetary value of the article expected on expiry of its life from a prospective buyer of the article, is called its scrap value or salvage value . For **SLM**, the formula used is:

Depreciation=(Cost-Scrap Value)/Life

For **DBM**, the formula used is:

Depreciation of the current year=Current Face Value * Rate

Where, initial face value is the cost and the next year's face value= Current year's Face Value – Current year's Depreciation.

For **SYDM**, the formula used is as shown below:

Illustrative example-5.5 using switch-Case logic–CONTD.

Depreciation for period I=(Cost*(Life-I+1))/(Sum of the period values in the Life)

The required code is shown below:

```
#Calculation of depreciation using any of a number of a number of available methods
cat(" Enter 1 for Straight Line Method, 2 for Declining Balance method, and 3 for Sum of years'digits method")
m=scan(n=1)
cat("Enter cost followed by Life")
v=scan(n=2)
cost=v[1];life=v[2]
print("Enter scrap/Salvage value")
sv=scan(n=1)
fv=cost
d=switch(m,((cost-sv)/life),as.numeric(readline(prompt="Rate?")),for(i in 1:life)v[i]=(cost*(life-i+1)*2)/(life*(life+1)))
if(m==1){
cat("The depreciation amount is: ",d,"for all the years of life")
} else if(m==2){ print(" YEAR DEPRECIATION")
for(y in 1:life){
dp=fv*d
cat("Year=",y," Depreciation= ",dp)
fv=fv-dp}
} else {
for(k in 1:life)

cat("Year=",k,"Depreciation=",v[k])}
```

Summary of the Lesson

The **case structure** is meant for selecting one of many available alternatives when the value of a case variable matches with one of the many alternatives. It can be replaced with an **if-else logic structure** where testing condition is always a condition of equality; but ; **the structure is preferred for elegant and compact coding** as illustrated through the solved problems.